

tmx Clipboard — Operator Guide

Revision: 3 **Last modified:** 2026-06-13T00:00:00Z **Authority:** vasic-digital tmux project **Maintainer:** milosvasic **Scope:** Operator-facing guide for the clipboard surface — **native terminal selection + right-click Copy as the default** (mouse off, works everywhere on Linux + macOS), the prefix `m` on-demand toggle for tmux mouse (wheel scrollback in TUIs + drag-copy to clipboard), prefix `P` paste-IN, plus the legacy keyboard copy-mode and modifier-drag fallbacks.

1. Quick recipe (TL;DR)

The shipped config ships with **mouse off** as the default, so the **terminal owns the mouse** — native click-drag selection (including multi-line), right-click → Copy, and native scroll all work the same as in any terminal, everywhere on Linux and macOS (iTerm2, Terminal.app, GNOME Terminal, WezTerm, ...). This is the recommended copy path.

Goal	Keystroke / mouse
Select & copy anything (default) — plain shell, Claude Code, vim, less, htop	native click-drag (multi-line OK) → Cmd-C / right-click → Copy
Paste OS clipboard back into the pane	native Cmd-V / right-click → Paste, or prefix + P
Wheel-scroll tmux's own scrollback inside a full-screen TUI	prefix + m (toggle tmux mouse ON) → wheel; prefix + m again to return to native
tmux drag-select that copies to the OS clipboard	prefix + m (mouse ON) → drag, release; prefix + m again to return to native
Copy without a mouse	prefix + [→ v → arrows / j / k → y
Default native scroll-back through history	wheel / trackpad / scrollbar (terminal-native, any device)

Replace prefix with your tmux prefix (default C-b).

2. Default architecture — native selection (mouse off), tmux mouse on demand (prefix m)

The forensic anchor for this section is the 2026-05-28 operator mandate: *"Selecting multiple lines and copying of them does not work. We MUST BE able to scroll vertically everywhere and copy / paste anything! Especially in Claude Code (claude command)!"*

Default: mouse off — the terminal owns the mouse

The shipped `~/.tmux.conf` sets **set -g mouse off** as the default:

```
set -g mouse off
```

With `mouse off`, tmux emits **zero** mouse-tracking enables — no CSI `?1000h` / `?1002h` / `?1006h` DECSET sequences reach the emulator. The emulator's native mouse is therefore completely unobstructed, so:

- **native click-drag selection works** — including **multi-line** selections;
- **right-click → Copy works** (and middle-click / Cmd-C);
- **native scroll / scrollbar works**.

...and all of it works **identically on Linux and macOS, on every emulator** (iTerm2, Terminal.app, GNOME Terminal, WezTerm, kitty, Alacritty, ...), **inside or outside a full-screen TUI** such as Claude Code / HelixCode. This is the recommended way to select and copy. To paste back in: native Cmd-V / right-click → Paste, or prefix `P` (see §4).

Why this is the default. A wire-level test (test 59) proved the root cause of the long-standing "can't select / copy" reports: when tmux had `mouse on`, it emitted the mouse-tracking DECSET enables, and those enables **suppressed** the emulator's native selection and right-click → Copy. With `mouse off` the native mouse is unobstructed, so the obvious gesture every operator already knows simply works.

On demand: prefix m — enable tmux's own mouse

Sometimes you want tmux's *own* mouse handling rather than the terminal's:

- to **wheel-scroll tmux's 50 000-line scrollbar inside a full-screen TUI** (the alternate screen has no scrollbar of its own, so with the default `mouse off` your wheel scrolls the terminal's buffer, not tmux's history); or
- to **drag-select inside tmux and have the selection copied straight to the OS clipboard** via the `@clip` pipeline (§3).

Press **prefix m** to toggle tmux mouse **ON**:

```
bind m set -g mouse \; display-message 'mouse #{?mouse,ON (tmux
selection: drag, or Shift-drag in apps),OFF (use native terminal
selection: drag + Cmd-C)}'
```

While mouse is ON, the shipped config's `WheelUpPane / WheelDownPane` overrides make the wheel always drive tmux copy-mode scrollbar, and a tmux drag selects and copies on release. Press **prefix m** again to return to native terminal selection + scrolling. The status line confirms the new state each time. (tmux toggles a flag/choice option when its value is omitted — see Sources verified.)

Modifier-drag fallback while tmux mouse is ON (Alt / Shift)

This subsection is a **fallback for the mouse on state only** — you do not need it for normal default-mode copying. When tmux mouse is ON and a TUI (Claude Code, vim, less, htop) is also requesting mouse tracking, an *unmodified* drag is forwarded to the app (so the app's own click behaviour keeps working) and therefore does not enter tmux copy-mode. tmux 3.6a supports modifier-prefixed mouse events (M- for Alt/meta, S- for Shift; see `man tmux` under "key names"), and the shipped config adds:

```
bind -n M-MouseDrag1Pane copy-mode -M
bind -n S-MouseDrag1Pane copy-mode -M
bind -T copy-mode-vi M-MouseDragEnd1Pane send-keys -X copy-pipe-
and-cancel "#{@clip}"
bind -T copy-mode-vi S-MouseDragEnd1Pane send-keys -X copy-pipe-
and-cancel "#{@clip}"
```

Effect: with tmux mouse ON, **hold Alt (Option on macOS) or Shift while you drag** to force tmux copy-mode entry regardless of what the TUI does with the mouse; release and the selection goes through the OS clipboard pipeline (@clip, §3). In practice, for everyday copying you are usually better off staying in the default mouse off state and using native selection — which needs no modifier at all.

OS	Modifier (only when tmux mouse is ON)	Why
Any OS / any terminal	Shift-drag	No mainstream terminal claims Shift as a bypass modifier, so Shift-mouse events reach tmux → S-MouseDrag1Pane. Works on macOS AND Linux.
macOS (Terminal.app, WezTerm, Ghostty, Alacritty)	Alt-drag (hold Option) also works	Option passes through to tmux → M-MouseDrag1Pane
Linux (gnome-terminal, konsole, xterm, kitty, foot, alacritty)	Alt-drag also works	Option/Meta forwarded to tmux on X11/Wayland
Termux on Android	long-press → "Select text" (terminal-native), then drag	Termux's gesture handler intercepts before tmux sees the event

iTerm2 note. With iTerm2's default *Option Key Sends = Normal*, iTerm2 treats Option-drag as its own native-selection bypass, so M-MouseDrag1Pane does not fire there; use Shift-drag if you are in the mouse on state. None of this matters in the default mouse off state, where iTerm2's native click-drag → Cmd-C simply works.

3. On the keyboard (no mouse at all)

Vi mode in copy-mode is enabled by default (set -g mode-keys vi in the shipped config).

prefix + [	enter copy-mode at the bottom of the buffer
prefix + PageUp	enter copy-mode AND page up in one motion
k / j	move up / down one line
C-u / C-d	half-page up / down
g / G	top / bottom of scrollback
/ / ?	search forward / backward
v	begin selection
V	line-mode selection (whole-line block)
C-v	rectangle (block) selection
y	copy through @clip → OS clipboard, exit copy-mode
Enter	same as y
q / Escape	exit copy-mode without copying

The y binding is what physically reaches pbpaste / wl-paste / xclip -o. Test 44 (scripts/tests/44_clipboard_copy_out_physical.sh) proves the literal y keystroke makes a marker show up on the OS clipboard end-to-end.

4. Paste OS clipboard INTO the current pane (v1.0.15)

set -g set-clipboard external is COPY-OUT only — OSC-52 is not standardised for paste-IN. v1.0.15 adds an explicit binding:

```
set -g @clip-read 'sh -c "command -v pbpaste >/dev/null 2>&1 &&
exec pbpaste; \
                                command -v wl-paste >/dev/null 2>&1 &&
exec wl-paste -n; \
                                command -v xclip >/dev/null 2>&1 && exec
xclip -o -selection clipboard; \
                                command -v termux-clipboard-get >/dev/
null 2>&1 && exec termux-clipboard-get; \
                                printf "\"\"'"
bind P run -b 'tmux load-buffer - <<< "${#@@clip-read}" \; tmux
paste-buffer -p'
```

- prefix + P (capital) reads the OS clipboard via the OS-adaptive @clip-read helper and pastes it into the current pane.

- `paste-buffer -p` enables **bracketed-paste** mode (per man `tmux`: "*If `-p` is specified, paste bracket control codes are inserted around the buffer if the application has requested bracketed paste mode*"). Apps that understand bracketed paste (bash, zsh, vim, neovim, Claude Code's prompt) treat the inserted text as a single literal block — no accidental command execution from a stray newline.

Lowercase prefix `+ p` is left bound to `tmux`'s previous-window so existing muscle memory is preserved.

5. On headless Linux (no clipboard tool installed)

`@clip` and `@clip-read` probe a closed set of tools in this order:

Order	Tool	Provides	Install
1	<code>pbcopy / pbpaste</code>	macOS native	preinstalled on Darwin
2	<code>wl-copy / wl-paste</code>	Wayland (sway, GNOME Wayland, KDE Wayland)	<code>apt install wl-clipboard</code>
3	<code>xclip</code>	X11 (Xorg)	<code>apt install xclip</code>
4	<code>termux-clipboard-set / termux-clipboard-get</code>	Android Termux	<code>pkg install termux-api</code>
5	OSC-52 (<code>set-clipboard external</code>)	any terminal that supports the OSC-52 escape	already enabled by the shipped config

If none of 1–4 are available AND the operator's terminal does not honour OSC-52, copy-OUT physical-proof test (T5 of test 44) **honestly SKIPS** with reason `no_clipboard_tool_reachable`. The binding chain (T3/T4) still verifies end-to-end inside `tmux`; only the final "did it land in `pbpaste`" hop is unverifiable on that topology. This is a §11.4.3 topology SKIP, not a PASS-bluff.

6. Troubleshooting

"Copy works in `tmux` but `pbpaste` returns the OLD clipboard." The `@clip` user option is set by the shipped `~/ .tmux.conf` — confirm it is present: `tmux show -gv @clip` inside a `tmx` session. If the output is empty, your `~/ .tmux.conf` is from before v1.0.14; re-run `bash scripts/setup.sh` to regenerate.

"I can't select / copy with the mouse." First make sure you are in the **default mouse off** state — then just use the **native terminal** selection (click-drag → `Cmd-C` / right-click → Copy), which works everywhere. Check the state with

`tmux show -gv mouse` (should print `off`); if it prints `on`, press prefix `m` to toggle back to native. Modifier-drag (next item) is only relevant when you have deliberately turned `tmux mouse` ON with prefix `m`.

"Alt-drag selects nothing; the app still gets the drag" (only matters in the mouse on state). Your terminal is capturing `Alt` before `tmux` sees it (common on Linux with `gnome-terminal`'s "Use Alt as Meta" preference disabled). Try Shift-drag instead — or simply press prefix `m` to return to native selection, which needs no modifier. As a permanent fix, check the terminal's "Pass Alt through to applications" / "Use Option as Meta key" setting.

"prefix + P does nothing." Either (a) `@clip-read` resolved to an empty probe — install one of `pbpaste` / `wl-paste` / `xclip` / `termux-clipboard-get`; or (b) the OS clipboard is genuinely empty. Verify with `tmux show -gv @clip-read` and then run the value in a shell to see what it returns.

"Mouse buttons behave weirdly after an aborted Alt-drag." Press `q` or `Escape` to exit copy-mode cleanly. `tmux` returns the pane to the app's mouse mode on copy-mode exit.

"I want to scroll tmux's scrollbar inside Claude Code, but the wheel scrolls the terminal instead." That is expected in the default `mouse off` state — the wheel drives the *terminal's* native scroll, and a full-screen TUI's alternate screen has no scrollbar of its own. To page back through `tmux`'s own 50 000-line history inside the TUI, press **prefix m** to toggle `tmux mouse` ON: the shipped config's `WheelUpPane` / `WheelDownPane` overrides then make the wheel drive `tmux` copy-mode scrollbar regardless of the app's mouse mode. Press prefix `m` again to return to native scrolling. (Confirm the overrides exist with `tmux show -g | grep -i WheelUp`; if absent, regenerate the config.)

7. Anti-bluff verification (positive evidence on every PASS)

Test	What it proves	Where
44 — clipboard copy-OUT physical	<code>y</code> keystroke through <code>@clip</code> reaches the OS clipboard; SKIPS honestly on headless	<code>scripts/tests/44_clipboard_copy_out_physical.sh</code>
45 — multi-line copy regression	keyboard <code>v</code> + arrows + <code>y</code> selects ≥ 3 lines; round-trip via <code>pbpaste</code>	<code>scripts/tests/45_multi_line_keyboard_copy.sh</code>
46 — paste-IN physical	prefix + <code>P</code> reads <code>@clip-read</code> and inserts into the pane	<code>scripts/tests/46_paste_in_physical.sh</code>
47 — mouse-surface confirmation	<code>tmux</code> observes the <code>M-</code> / <code>S-</code> modifier on drag-start	<code>scripts/tests/47_modifier_mouse_surface.sh</code>

Test	What it proves	Where
48 — modifier- drag binding chain	M–MouseDown1Pane + M– MouseDown1Pane resolves to @clip end-to-end	scripts/tests/ 48_modifier_drag_binding_chain.sh

Each test follows Constitution §101 (positive captured evidence) and §11.4.69 (sink-side / downstream evidence — pbbaste IS the sink). Paired mutations M44 / M46 / M48 in `meta_test_false_positive_proof.sh` strip the relevant binding and assert each test FAILs (CAUGHT + FEATURE RESTORED).

8. Cross-references

- `scripts/tmux.conf.template` — the canonical source of all bindings cited above
- `docs/guide/README.md` §5 — operator-command reference
- `docs/scrolling/README.md` — sister guide for wheel / touch scrolling in Claude Code
- `Fixed.md` A35 (v1.0.14 copy-OUT) + A37 (v1.0.15 multi-line + paste-IN + modifier-drag)
- `CHANGELOG.md` v1.0.14 + v1.0.15 sections

Sources verified 2026-06-13

- **tmux upstream man page** — <https://man.openbsd.org/tmux.1> (re-verified 2026-06-13 via WebFetch; OpenBSD ships the canonical upstream `tmux.1`). Confirmed for the mouse off default architecture: `set-clipboard external` "will attempt to set the terminal clipboard but ignore attempts by applications to set tmux buffers" (the OSC-52 copy-OUT path used by @clip); and a flag/choice option set with its value omitted toggles its value (validates `bind m set -g mouse` as the prefix `m` toggle). The claim that mouse off emits no mouse-tracking DECSET enables (`?1000h` / `?1002h` / `?1006h`), so the emulator's native selection / right-click → Copy / scroll is unobstructed, is proven at the wire level **in this repo** by test 59 (`scripts/tests/59_*.sh`), which is the load-bearing authority for the default-architecture claims above. The canonical binding source is `scripts/tmux.conf.template`.

Sources verified 2026-05-29

- **tmux upstream man page** — <https://man.openbsd.org/tmux.1> (re-verified 2026-05-29 via WebFetch, OpenBSD ships the upstream `tmux` man page). Confirmed for the v1.0.17 prefix `m` addition: modifier prefixes "Ctrl keys may be prefixed with C– or ^, Shift keys with S– and Alt (meta) with M–" (validates `S–MouseDown1Pane` / `M–MouseDown1Pane`); `–M` "passes through a mouse event (only valid if bound to a mouse key binding, see MOUSE SUPPORT)"; flag/choice options "can be omitted ... to toggle its value" (validates `set -g mouse`

with no value as a toggle); `#{?condition,true,false}` is the standard FORMATS conditional used in the `display-message` status hint.

Sources verified 2026-05-28

- **tmux 3.6a man page** — `man tmux` on the host (Mistborn Darwin arm64, tmux 3.6a from `tmux/build-darwin/bin/tmux`). Confirmed: S- / M- modifier prefixes for keys including mouse events ("Ctrl keys may be prefixed with C- or ^, Shift keys with S- and Alt (meta) with M-"); `MouseDrag1 / MouseDragEnd1` events listed in `MOUSE SUPPORT`; `paste-buffer -p` documented as "paste bracket control codes are inserted around the buffer if the application has requested bracketed paste mode"; user options prefixed with @ are arbitrary and string-typed.
- **tmux upstream man page mirror** — <https://man.openbsd.org/tmux.1> (verified 2026-05-28; OpenBSD ships the upstream tmux man page).
- **Anthropic Claude Code documentation** — <https://code.claude.com/docs/en/> (verified 2026-05-28; the public docs do not currently publish a tmux-integration page, so the alt-screen + mouse-tracking behaviour cited above is documented here from direct observation against the Claude Code CLI v2.x — the upstream binary that ships as `bin/claude.exe` Mach-O on macOS and is exercised in `scripts/tests/47..48`).
- **Synthetic alt-screen surrogate** — `scripts/tests/helpers/synthetic_alt_screen_app.py` (this repo) substitutes for Claude Code in CI to avoid §11.4.98 OAuth/interactive flake while still exercising the exact alt-screen + mouse-tracking surface.